

NAME : Nevetha S K
EMAIL ID : nevethasenthilkumar@gmail.com

CODING ASSESSMENT - Apache Airflow – Features And Pipeline Building

Introduction

Apache Airflow is an open-source platform used to programmatically author, schedule, and monitor workflows. It allows data engineers to build data pipelines that are reliable, scalable, and easy to monitor.

Key Features of Apache Airflow

- 1. Dynamic Workflow Generation**
 - Workflows are defined as Python code.
 - Allows parameterization and dynamic DAG creation.
- 2. Scalable & Distributed Execution**
 - Uses Celery, Kubernetes, or other executors for distributed execution.
 - Can handle large-scale workflows with parallelism.
- 3. Rich User Interface (UI)**
 - Provides a web UI to visualize DAGs (Directed Acyclic Graphs).
 - Easy monitoring of tasks, logs, and status.
- 4. Extensible Architecture**
 - Integrates with multiple systems (AWS, GCP, Azure, Hadoop, SQL, etc.).
 - Custom operators and hooks can be created.
- 5. Scheduling & Orchestration**
 - Built-in scheduler for periodic or event-driven execution.
 - Supports retries, SLAs, and alerts.

6. Task Dependency Management

- DAGs enforce task execution order.
- Allows conditional branching and task retries.

7. Community & Ecosystem

- Large open-source community.
- Hundreds of pre-built operators and providers.

Steps to Build a Pipeline in Airflow

Step 1: Define a DAG File

1. Open the Airflow project directory.
2. Locate the dags folder.
3. Create a new Python file (e.g., example_dag.py).
4. Import the required Airflow libraries such as DAG, operators, and datetime.
5. Define the default arguments like owner, start date, retries, etc.
6. Initialize a DAG object with a unique DAG ID and scheduling interval.
7. Define tasks using available operators (e.g., BashOperator, PythonOperator).
8. Set dependencies between tasks using `>>` or `<<`.

Step 2: Place the DAG in the Correct Folder

- Save the Python file in the dags directory.
- Ensure the file has no syntax errors, as Airflow scans this folder to detect DAGs.

Step 3: Start Airflow Services

1. Start the Airflow web server.
2. Start the Airflow scheduler.

3. Ensure both services are running in the background.

Step 4: Access Airflow UI

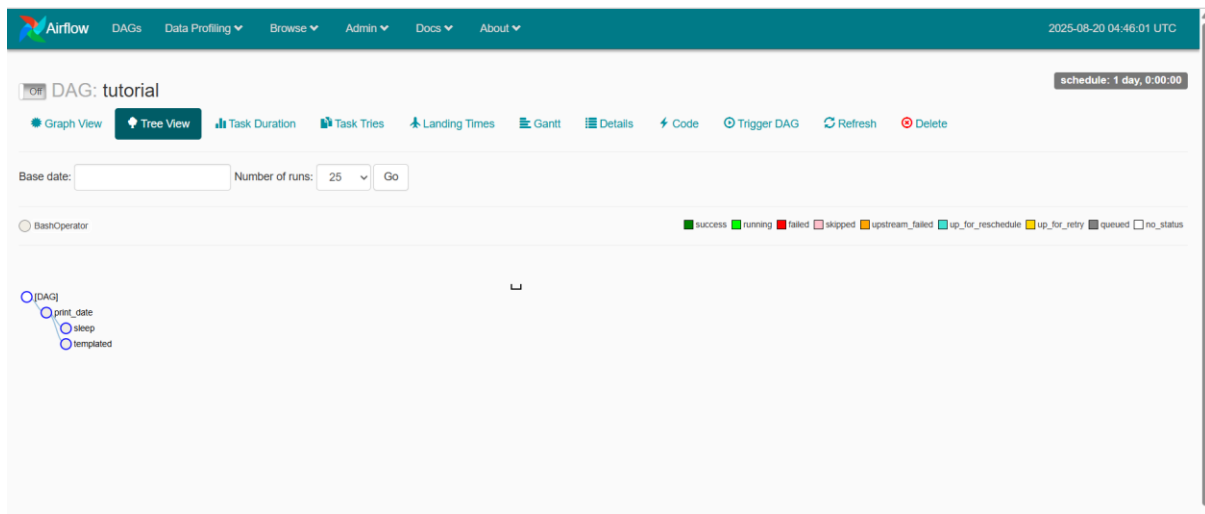
1. Open a browser and go to the Airflow web interface (default: `http://localhost:8080`).
2. Log in using the default or configured credentials.
3. Navigate to the **DAGs page**, where all available pipelines will be listed.

Step 5: Enable and Run a DAG

1. Locate the newly created DAG in the list.
2. Toggle the switch to enable the DAG.
3. Trigger the DAG manually or wait for the scheduled run.

Step 6: Monitor Pipeline Execution

1. Click on the DAG name to view details.
2. Use the **Graph View** to visualize task dependencies.
3. Use the **Tree View** to track execution history.
4. Open **Task Logs** to view real-time logs of each task execution.



Viewing & Monitoring Pipelines in Airflow

1. DAGs Dashboard

- Shows list of available DAGs.
- Provides controls to pause/resume, trigger, or delete DAG runs.

2. Graph View

- Displays DAG as a **Directed Acyclic Graph**.
- Visualizes task dependencies and execution flow.

3. Tree View

- Shows tasks by execution date.
- Helps track success, failure, or skipped tasks.

4. Task Instance Details

- Logs, execution time, and retries per task.
- Option to re-run failed tasks.

5. Gantt Chart View

- Timeline view for tasks.
- Helps identify bottlenecks and parallel execution.

6. Code View

- Displays the underlying Python code of the DAG.
- Useful for debugging and quick edits.

Conclusion

Apache Airflow provides a powerful, flexible, and scalable way to build and manage data pipelines. With its dynamic DAGs, rich monitoring interface, and strong community support, it has become a standard tool in modern data engineering workflows.